FABRYKA STRON - ARCHITEKTURA OPERACYJNA V3

Kompletny plan wdrożenia od zera do działającego systemu produkcji i utrzymania stron lokalnych

Wersja	3.0
Przeznaczenie	Jedno źródło prawdy dla budowy, publikacji i utrzymania fabryki stron
Model pracy	Półautomat: AI generuje i przyspiesza, Ty zatwierdzasz i publikujesz
Docelowa skala	Do około 30 stron miesięcznie przy jednym operatorze
Data	2026-04-10

Jak używać tego dokumentu

To nie jest tylko opis technologii. To dokument operacyjny. Najpierw czytasz rozdziały 1-4, potem wdrażasz według rozdziału 15, a na co dzień pracujesz według rozdziałów 8-14. Dokument zakłada jednoosobową obsługę, brak dostępu klienta do panelu i nacisk na jakość, utrzymanie oraz przewidywalność procesu.

Spis treści roboczy

- 1. Cel dokumentu i założenia biznesowe
- 2. Różnica między v2 a v3
- 3. Zasady projektowe v3
- 4. Docelowy stack i rola każdego elementu
- 5. Architektura logiczna systemu
- 6. Source of Truth: model danych i odpowiedzialności
- 7. Klasy zmian i zasady publikacji
- 8. Change Center: centrum operacyjne zmian
- 9. Versioning, snapshoty i rollback
- 10. Job Engine i automatyzacje
- 11. AI Layer i architektura promptów
- 12. Design System i unikalność wizualna
- 13. Folder structure, repo i standardy kodu
- 14. Bezpieczeństwo, backup, QA i monitoring
- 15. Plan wdrożenia od zera
- 16. SOP-y dla najczęstszych zmian
- 17. Załączniki: schema v3, status machine, checklisty

1. Cel dokumentu i założenia biznesowe

Ten dokument scala Twoje dotychczasowe decyzje z wersji v2 i rozwija je do poziomu operacyjnego v3. Celem jest zbudowanie jednego systemu, który pozwala nie tylko wdrażać strony lokalnych biznesów, ale także bezpiecznie nimi zarządzać, aktualizować je po publikacji, kontrolować jakość oraz utrzymać porządek przy rosnącej liczbie klientów.

Założenia wejściowe są konkretne: strony dla lokalnych biznesów w Polsce, głównie usługowych i specjalistycznych, model półautomatyczny, jeden operator, brak dostępu klienta do panelu, nacisk na SEO, unikalność i łatwość utrzymania.

> **Najważniejsza zmiana mentalna v3**
>
> System ma przestać być tylko generatorem stron. Ma stać się systemem operacyjnym do produkcji, utrzymania i bezpiecznej aktualizacji stron.

Założenia biznesowe

- 1 operator (Ty) obsługuje cały proces.
- Klient zgłasza zmiany, ale nie ma dostępu do panelu.
- Priorytetem jest jakość strony i możliwość utrzymania, a nie maksymalna automatyzacja za wszelką cenę.
- Docelowo system ma obsługiwać około 30 wdrożeń miesięcznie bez chaosu procesowego.
- Treść ma być unikalna, prawdziwa i oparta na danych dostarczonych przez klienta albo zatwierdzonych przez Ciebie.
- Zwykłe poprawki muszą być szybkie, a zmiany strukturalne muszą przechodzić przez preview i kontrolę jakości.

Definicja sukcesu

Obszar	Co oznacza sukces	Miara praktyczna
Produkcja	Nowy klient przechodzi od briefu do preview bez improwizacji.	Stały workflow i checklisty.
Utrzymanie	Telefon, godziny, FAQ i zdjęcia da się zmienić bez dotykania kodu.	Zmiany operacyjne do obsłużenia w kilka minut.
Bezpieczeństwo	Każda publikacja ma historię, diff i rollback.	Brak strachu przed deployem.
Skalowanie	Dodanie kolejnego klienta nie wymaga przebudowy systemu.	Powtarzalne template family i job queue.
Jakość	Strony nie wyglądają jak klony i nie tworzą doorway spam.	Kontrola page worthiness i visual profile.

2. Różnica między v2 a v3

v2 dobrze opisuje fundament: stack, bazę, dashboard, chaining promptów, build i deploy. v3 nie zastępuje tego planu - ono go wzmacnia. Największa różnica polega na tym, że v3 dodaje warstwę operacyjną, dzięki której system nie rozsypuje się po pierwszych wdrożeniach i pierwszych zgłoszeniach typu „zmień numer telefonu” albo „dodaj usługę”.

Obszar	v2	v3
Publikacja	Zapis -> rebuild	Klasyfikacja zmiany -> wersja -> job -> deploy
Zmiany po wdrożeniu	Prosty request i ręczna reakcja	Change Center, typy zmian, preview i rollback
Treść	Bieżący rekord w tabeli	Wersje draft/approved/published
Build	Akcja wywołana z dashboardu	Job engine z logami i stanami
Historia	System logs	System logs + content diffs + deployment snapshots
Unikalność wizualna	Template branżowy	Template family + design profile + variants
Bezpieczeństwo zmian	Rebuild po zapisie	Preview diff + snapshot + rollback

Wniosek

v2 opisuje „jak zbudować stronę”. v3 opisuje „jak zbudować i prowadzić całą fabrykę stron bez chaosu”.

3. Zasady projektowe v3

1. Jedno źródło prawdy: wszystkie dane klienta, treści, media, requesty, wersje i logi muszą żyć w jednym systemie - Supabase.
2. Aplikacja steruje automatyzacją: rebuildy, generowanie treści i workflow nie są ukryte w bazie.
3. Nie każda zmiana jest taka sama: szybkie poprawki, edycje treści i zmiany strukturalne mają osobne ścieżki publikacji.
4. Każda istotna publikacja musi być odwracalna: snapshoty i rollback są obowiązkowe.
5. AI ma przyspieszać i proponować, ale nie ma być jedynym źródłem prawdy.
6. Unikalność nie może wynikać z chaosu. Powinna wynikać z kontrolowanych template family, variantów i danych klienta.
7. Najpierw odporność operacyjna, potem „fancy” dodatki SEO.

Czego nie robimy

- Nie budujemy „magicznego autopilota”, który sam publikuje wszystko bez Twojej zgody.
- Nie wrzucamy każdej zmiany do kodu ręcznie.
- Nie trzymamy treści jako luźnych plików Markdown poza source of truth.
- Nie projektujemy każdej strony od zera - budujemy powtarzalny, kontrolowany system.
- Nie generujemy location pages bez wartości, dowodów lokalnych i sensu biznesowego.

4. Docelowy stack i rola każdego elementu

Warstwa	Technologia	Rola	Uwagi operacyjne
Strony klientów	Astro SSG	Frontend live, nacisk na szybkość i prostotę	Czyta dane w build time ze snapshotu lub source of truth.
Dashboard	Astro SSR	Panel operatora i centrum dowodzenia	To tutaj żyje Change Center, Content Studio i Build Center.
Baza i CMS	Supabase	Source of Truth dla danych, treści, wersji i logów	Zawiera też Storage i warstwę operacyjną.
AI treściowe	Anthropic API	Generowanie i przepisywanie treści, altów, FAQ, draftów	Zawsze przez chaining i walidację.
Agent deweloperski	Claude Code	Buduje szablony, komponenty, refaktory i standardy	Nie odpowiada za finalną prawdziwość treści.
Hosting i deploy	Vercel	Preview i produkcja	Deploye sterowane przez aplikację i job queue.
Storage mediów	Supabase Storage	Logo, zdjęcia usług, galerie, assety klienta	Bucket prywatny lub publiczny zależnie od strategii.
Repozytoria	GitHub	Kod dashboardu, template'ów i projektów klientów	Repo per template lub repo centralne + projekty buildowe.

Zasada wyboru technologii

Nie zmieniamy stacku tylko dlatego, że da się coś zrobić inaczej. Operacyjnie problemem nie jest technologia, tylko brak wersjonowania, jobów, snapshotów i kontroli zmian.

5. Architektura logiczna systemu

v3 składa się z siedmiu warstw. Każda z nich odpowiada za inny rodzaj problemu. Dzięki temu nie mieszasz danych biznesowych, treści, logiki operacyjnej i buildów w jednym miejscu.

1. Source of Truth Layer

Klienci, usługi, lokalizacje, treści, media, ustawienia, requesty, logi.

2. Content Layer

Strony, sekcje, FAQ, posty, metadata i assety używane do publikacji.

3. Operations Layer

Change Center, klasyfikacja zmian, job queue, snapshoty, rollback.

4. AI Layer

Chaining promptów, generowanie draftów, rewrite, alt-text, walidacja.

5. Build Layer

Build preview, build publish, deploy, indexing, smoke tests.

6. Design System Layer

Template family, design profile, variant manifests, bloki UI.

7. Audit & Feedback Layer

System logs, content diffs, deployment history, monitoring, indeksacja.

> **Ważne rozdzielenie**
>
> v2 skupiało się głównie na warstwach 1, 2, 4 i 5. v3 dokłada warstwę 3 i 7, które robią z systemu prawdziwy produkt operacyjny.

6. Source of Truth: model danych i odpowiedzialności

W v3 Supabase nadal jest centrum systemu, ale struktura musi być bardziej świadoma. Dzielimy dane na cztery grupy: dane biznesowe, treści publikacyjne, operacje i design. To zapobiega sytuacji, w której jedna tabela odpowiada jednocześnie za content, workflow i deploy.

6.1 Grupy danych

Grupa	Przykładowe tabele	Za co odpowiada
Biznes	clients, site_config, services, locations, asset_library	To, co klient realnie robi, sprzedaje i pokazuje.
Publikacja	pages, client_posts, *_versions, page_section_variants	To, co trafia na stronę i w jakiej wersji.
Operacje	change_requests, jobs, job_steps, deployments, deployment_snapshots	Jak zmiany przechodzą od zgłoszenia do live.
Audit i feedback	system_logs, content_diffs, indexing_events, performance_snapshots	Historia, kontrola jakości, wnioski po publikacji.

6.2 Tabele, które zostają z v2

- clients - główna encja klienta.
- site_config - kontakt, marka, feature flags, dane publikacyjne globalne.
- services - lista usług klienta jako encje biznesowe.
- locations - lokalizacje, miasta, service areas.
- pages - strony stałe typu home, about, contact.
- client_posts - wpisy eksperckie / blogowe.
- system_logs - logi zdarzeń.

6.3 Tabele, które dodajemy w v3

- change_requests - formalne centrum zmian.
- jobs i job_steps - stan, kolejka i postęp akcji długotrwałych.
- service_versions, location_versions, page_versions, post_versions - wersjonowanie treści.
- deployment_snapshots - zamrożony stan publikacji.
- deployments - historia preview, publish i rollbacków.
- design_profiles - profil wizualny klienta.
- page_section_variants - mapowanie wariantów sekcji użytych na stronie.
- content_diffs - porównania zmian przed publikacją.
- indexing_events i performance_snapshots - podstawowa pętla po publikacji.

6.4 Zasada danych live vs strukturalnych

Nie wszystkie dane są równie „ciężkie”. Numer telefonu i godziny to dane globalne. Nowa usługa i nowa lokalizacja to zmiany strukturalne. W source of truth te dwie klasy danych muszą być obsługiwane inaczej, bo wpływają na różne warstwy systemu.

Praktyczna reguła

Jeśli zmiana wpływa tylko na treść istniejącego route’a, możesz obsłużyć ją szybciej. Jeśli wpływa na routing, sitemapę, architekturę linkowania albo generuje nową stronę - to jest zmiana strukturalna i wymaga preview.

7. Klasy zmian i zasady publikacji

To jest jeden z kluczowych elementów v3. Bez klasyfikacji zmian system będzie albo zbyt ciężki, albo zbyt niebezpieczny. Każda zmiana musi zostać przypisana do jednej z czterech klas.

Klasa zmiany	Przykłady	Wymagania	Tryb publikacji
quick_edit	Telefon, e-mail, godziny, CTA URL, social media	Brak AI, brak preview	instant_rebuild
content_edit	Opis usługi, FAQ, o nas, opinie, zdjęcia	Draft + diff + akceptacja	draft_then_publish
structural_edit	Nowa usługa, lokalizacja, zmiana sluga, nowa strona	AI lub ręczne przygotowanie + preview	preview_then_publish
design_edit	Zmiana wariantu sekcji, fontów, visual profile, migracja template'u	Preview obowiązkowy, często kilka stron naraz	preview_then_publish

7.1 Publish modes

- instant_rebuild - szybki zapis i rebuild bez preview.
- draft_then_publish - zapis wersji roboczej, diff, akceptacja, potem produkcja.
- preview_then_publish - preview deployment, review, dopiero potem publish.
- scheduled_publish - opcjonalny tryb na przyszłość, np. promocje sezonowe albo content loop.

7.2 Reguła bezpieczeństwa

Jeżeli masz wątpliwość, czy dana zmiana może wpłynąć na routing, indexację, linkowanie albo wygląd wielu stron naraz - klasyfikujesz ją wyżej, a nie niżej. Lepiej wysłać zmianę do preview niż publikować ją „na żywo" w ciemno.

8. Change Center: centrum operacyjne zmian

Change Center to nowy rdzeń dashboardu v3. To tu trafia każda zmiana po wdrożeniu: od prostych poprawek po nowe usługi i redesign sekcji.

8.1 Jak działa Change Center

8. Tworzysz request lub importujesz go z rozmowy / formularza / telefonu.
9. System nadaje request_type i change_class.
10. Request trafia do triage.
11. Jeśli trzeba, tworzony jest job AI albo draft ręczny.
12. Powstaje wersja treści lub przygotowanie zmiany strukturalnej.
13. Review i akceptacja.
14. Publish albo preview + publish.
15. Logi, snapshot i ewentualne indexing event.

8.2 Minimalny model requestu

Pole	Przykład	Znaczenie operacyjne
request_type	change_phone, add_service	Słownik gotowych typów requestów.
change_class	quick_edit / structural_edit	Steruje ścieżką publikacji.
target_table + target_record_id	site_config / service 123	Wskazuje, czego dotyczy zmiana.
requires_ai	true / false	Czy trzeba uruchamiać generowanie lub rewrite.
requires_preview	true / false	Czy publikacja musi przejść przez preview.
priority	low / normal / high	Steruje kolejnością i komunikacją.
status	new / triaged / awaiting_review ...	Stan requestu w centrum zmian.

8.3 Widoki w Change Center

- Inbox - wszystkie nowe zgłoszenia.
- To triage - trzeba przypisać klasę zmiany.
- Draft in progress - zmiana jest przygotowywana.
- Awaiting review - czeka na Twoją akceptację.
- Ready to publish - wszystko gotowe do deployu.
- Completed - wdrożone i zamknięte.
- Rolled back / rejected - historia wyjątków.

9. Versioning, snapshoty i rollback

v3 zakłada, że nie nadpisujesz po prostu treści w miejscu. Każda istotna treść dostaje wersje. Każda publikacja dostaje snapshot. Każdy zły deploy można odwrócić.

9.1 Wersjonowanie treści

- service_versions - każda publikowalna wersja opisu usługi.
- location_versions - każda wersja strony lokalizacyjnej.
- page_versions - home, about, contact, landingi specjalne.
- post_versions - wpisy eksperckie i blogowe.

Każda wersja ma minimum: version_no, status, source, body, metadata, created_at, approved_at, published_at.

9.2 Snapshoty publikacyjne

Snapshot to zamrożony stan klienta w momencie publikacji. Zawiera dokładnie to, co ma pójść do builda: aktywne wersje treści, globalny config, media manifest, design profile, template version i routing manifest.

> **Po co snapshot**
>
> Snapshot rozwiązuje trzy problemy naraz: rollback, audyt i przewidywalność builda. Zamiast budować „z żywej bazy”, możesz budować z zamrożonego zestawu danych.

9.3 Rollback

16. Wybierasz poprzedni deployment albo snapshot.
17. System tworzy job rollback.
18. Build opiera się na starym snapshotcie, nie na „cofnięciu” bieżących rekordów.
19. Powstaje nowy deployment typu rollback.
20. Logujesz przyczynę i wynik w system_logs.

9.4 Preview diff

Przed publish powinieneś zobaczyć nie tylko preview URL, ale także różnicę: które strony się zmieniły, jakie meta zostały ruszone, jakie sekcje zostały dodane lub usunięte i czy zmieniły się CTA. To minimalizuje ryzyko przypadkowych zmian.

10. Job Engine i automatyzacje

W v2 dużo akcji było wywoływanych bezpośrednio z kliknięcia. W v3 wszystko, co może trwać dłużej niż chwilę albo może się nie udać, powinno być jobem.

10.1 Typy jobów

- generate_site
- generate_service
- generate_location
- generate_post
- rewrite_section
- upload_image
- generate_image_alt
- build_preview
- build_publish
- request_indexing
- rollback

10.2 Statusy jobów

- queued
- running
- waiting_for_input
- failed
- completed
- canceled

10.3 Korzyści z job engine

- Nie tracisz procesu po zamknięciu karty.
- Masz retry i logi błędów.
- Możesz uruchamiać procesy sekwencyjnie lub etapami.
- Masz jeden spójny model postępu dla generowania, buildów i rollbacków.
- Łatwiej debugować problemy z AI, deployem albo obrazkami.

10.4 Struktura joba

Każdy job powinien mieć: job_type, client_id, request_id, trigger_source, payload_json, status, progress, result_json, error_message, started_at, finished_at. Przy bardziej złożonych jobach dodajesz job_steps, żeby widzieć, który etap był problematyczny.

11. AI Layer i architektura promptów

v3 utrzymuje chaining, ale porządkuje odpowiedzialności. AI ma działać jako zestaw wyspecjalizowanych narzędzi, a nie jedna wielka „magiczna” funkcja.

11.1 Zestaw agentów logicznych

Agent	Wejście	Wyjście	Rola
Brief Normalizer	Brief, dane klienta, załączniki	Ustrukturyzowane dane wejściowe	Porządkuje input i wykrywa braki.
Keyword Strategist	Usługi, lokalizacje, seed listy	Klastry i intencje	Buduje mapę tematów i pytań.
Page Worthiness	Klastry, lokalizacje, proofy	Lista stron do budowy	Oddziela strony warte tworzenia od zbędnych.
Section Composer	Blueprint strony i dane klienta	Draft sekcji i metadata	Pisze treść w strukturze bloków.
Critic	Drafty, istniejące strony, reguły	Lista poprawek i alertów	Wyłapuje generyki, overlap i ryzyka.
Builder Operator	Zatwierdzony snapshot	Build output	Składa i publikuje projekt, nie wymyśla treści.

11.2 Zasady promptów

- Każdy prompt ma jasno określony input, output i reguły prawdziwości.
- Każdy prompt powinien zwracać przewidywalny format - najlepiej JSON lub ściśle określony blok markdown.
- Nie generujesz wszystkiego na raz. Rozbijasz architekturę i treść na małe kroki.
- Reguły prawdziwości są wspólne dla wszystkich promptów i trzymane w jednym miejscu.
- Prompty branżowe i prompty per page type są składane warstwowo.

11.3 Gdzie AI NIE powinno decydować samodzielnie

- Czy dana usługa ma dostać osobną stronę bez local proof i sensu biznesowego.
- Czy lokalizacja zasługuje na dedykowaną stronę.
- Czy claim jest dozwolony lub bezpieczny regulacyjnie.
- Jaką finalną zmianę wypchnąć na produkcję bez Twojej akceptacji.

12. Design System i unikalność wizualna

v3 utrzymuje template'y branżowe, ale dopina do nich dodatkową warstwę kontroli wizualnej. Dzięki temu strony są różne, ale nadal łatwe w utrzymaniu.

12.1 Trzy poziomy designu

- Template family - np. medical, trade, legal, beauty, universal.
- Design profile - ustawienia klienta: font pair, CTA style, density, radius, shadows, rytm sekcji.
- Page section variants - konkretne warianty hero, FAQ, kart usług, review i CTA na danej stronie.

12.2 Minimalny design profile

Pole	Przykład	Wpływ
template_family	medical	Określa podstawowe bloki i layout archetype.
font_pair	DM Serif + DM Sans	Nadaje charakter typografii.
cta_style	soft / urgent / premium	Wpływa na brzmienie i wygląd CTA.
section_rhythm	airy / compact	Kontroluje zagęszczenie i odstępy.
radius_style	soft / squared	Nadaje charakter kartom i przyciskom.
motion_level	none / low / medium	Określa zakres animacji.
imagery_style	clinical / practical / elegant	Steruje doborem zdjęć i kompozycją.

12.3 Co realnie daje unikalność

- Różny rytm sekcji i hierarchia informacji.
- Różne warianty komponentów, a nie tylko inny kolor.
- Różne CTA i ich rozmieszczenie.
- Prawdziwe zdjęcia klienta i właściwe proof blocks.
- Dane klienta i jego realne przewagi wkomponowane w layout.

Czego nie robić

Nie próbuj robić każdego klienta jako pełny custom design od zera. Koszt utrzymania i chaos szybko Cię dogonią. Unikalność musi być kontrolowana.

13. Folder structure, repo i standardy kodu

Repozytoria mają wspierać operacyjny porządek. Najbezpieczniejszy model to jeden dashboard jako centrum dowodzenia oraz katalog template'ów i buildów klientów w kontrolowanej strukturze.

13.1 Główna struktura projektu

```
fabryka-stron/
  dashboard/                # panel operatora
  templates/                # template families
  scripts/                  # utility i build scripts
  workers/                  # job workers
  prompts/                  # system prompty i prompty per typ
  shared/                   # diffy, klasyfikatory, helpery
  projects/                 # zbudowane projekty klientów
  docs/                     # runbooki i checklisty
```

13.2 Dashboard v3 - moduły

- Client Hub
- Change Center
- Content Studio
- Preview & Diff
- Build & Deploy
- Design Studio
- System Logs / Audit

13.3 Standardy dla repo klienta

- Każdy projekt klienta ma przypisany template family i template version.
- Treść na build pobierana jest z zatwierdzonego snapshotu lub z warstwy publikacyjnej, nie z przypadkowych plików.
- Każdy projekt ma minimalny README z informacją o klientu, domenie, template version i komendach buildowych.
- Każdy projekt ma checklistę QA i standardy metadata/schema.
- Zmiany globalne w blokach idą przez template'y, nie przez ręczne poprawki w wielu projektach naraz.

14. Bezpieczeństwo, backup, QA i monitoring

14.1 Dostęp i role

Ponieważ system ma jednego operatora, nie potrzebujesz złożonego RBAC. Ale nadal potrzebujesz dobrych nawyków bezpieczeństwa.

- Jeden operator w dashboardzie.
- Service keys tylko po stronie serwera.
- RLS dla tabel eksponowanych do aplikacji.
- Sekrety trzymane w envach Vercela i bezpiecznych ustawieniach serwera.
- Oddzielne klucze i hooki per środowisko, jeśli wprowadzisz staging lub osobne preview workflow.

14.2 Backup

- Snapshoty publikacyjne jako podstawowy mechanizm odtworzenia publikacji.
- Historia deploymentów.
- Kopie repozytoriów w GitHub.
- Regularny eksport lub backup najważniejszych tabel operacyjnych i treściowych.

14.3 QA

- Automatyczna walidacja treści i metadata.
- Kontrola brakujących CTA, FAQ, links i danych kontaktowych.
- Smoke test po deployu.
- Manualny review preview przed każdą zmianą strukturalną.
- Checklista jakości wizualnej dla nowych template family i zmian designowych.

14.4 Monitoring

- Logi buildów i deployów.
- Status indexing events.
- Rejestr błędów i failed jobs.
- Podstawowy uptime i informacja, kiedy ostatni build był udany.
- W kolejnej fazie: performance snapshots i bardziej zaawansowane SEO monitoring.

15. Kompletny plan wdrożenia od zera

Poniżej masz praktyczny plan wdrożenia od zera, rozpisany w logicznej kolejności. Kluczowe jest to, żeby nie budować wszystkiego naraz. Najpierw fundament, potem panel, potem workflow zmian, potem dopiero warstwa zaawansowanego SEO i automatyzacji.

Faza 0 - Przygotowanie środowiska

- Załóż Supabase i Vercel project dla dashboardu.
- Uporządkuj repozytorium główne fabryki stron.
- Zdefiniuj konwencje nazw, branchy i folderów.
- Wybierz jeden template family jako pierwszy: universal lub trade.

Faza 1 - Fundament danych i pierwszy live build

- Utwórz core tables z v2: clients, site_config, services, locations, pages, client_posts, system_logs.
- Dodaj asset_library i bucket client_media.
- Postaw pierwszy template Astro czytający dane klienta z Supabase.
- Ręcznie wpisz testowego klienta i zbuduj testową stronę.

Faza 2 - Dashboard operacyjny MVP

- Client list, client detail, basic edit screen.
- Brief intake i zapis danych do Supabase.
- Generate-site chaining jako job.
- Prosty build i deploy z dashboardu.
- Walidacja treści i podstawowe system_logs.

Faza 3 - v3 Operations Layer

- Dodaj change_requests, jobs, job_steps.
- Dodaj service_versions, location_versions, page_versions, post_versions.
- Zbuduj Change Center i klasyfikację zmian.
- Dodaj deployments i deployment_snapshots.
- Uruchom preview diff i rollback.

Faza 4 - Design i skalowanie template'ów

- Dodaj design_profiles i page_section_variants.
- Rozszerz template families do pozostałych branż.
- Spisz katalog bloków, wariantów i zasad ich użycia.
- Wdróż checklistę spójności wizualnej.

Faza 5 - SEO i automatyzacje drugiego rzędu

- Indexing events i integracja z Google Indexing API tam, gdzie ma sens.
- Content loop jako opcja, nie fundament.
- Performance snapshots i monitoring jakości wdrożeń.
- Lepsze asset workflows, alt-text i optymalizacja obrazów.

15.1 Minimalna kolejność budowy

21. Source of Truth i pierwszy template.
22. Dashboard MVP.
23. Generate-site i build job.
24. Versioning.
25. Change Center.
26. Snapshots i deployments.
27. Rollback.
28. Design profiles.
29. Pozostałe template family.
30. SEO automatyzacje.

16. SOP-y dla najczęstszych zmian

SOP 1 - Zmiana numeru telefonu

- Utwórz request typu change_phone.
- Klasyfikacja: quick_edit.
- Zmień wartość w site_config.phone.
- Zapisz content diff.
- Utwórz snapshot przed zmianą albo oznacz deployment source.
- Uruchom build_publish.
- Sprawdź stronę kontaktową, header i CTA po deployu.
- Zamknij request i dopisz log.

SOP 2 - Dodanie nowej usługi

- Utwórz request add_service.
- Klasyfikacja: structural_edit.
- Dodaj rekord service jako encję biznesową.
- Uruchom generate_service.
- Powstaje service_version draft.
- Zrób review i poprawki.
- Zbuduj preview snapshot i build_preview.
- Po akceptacji: build_publish + indexing event.

SOP 3 - Dodanie FAQ do istniejącej usługi

- Utwórz request add_faq lub rewrite_service_section.
- Klasyfikacja: content_edit.
- Stwórz draft nowej wersji usługi.
- Sprawdź diff i publikuj bez pełnego preview, jeśli polityka projektu na to pozwala.
- W przypadku bardziej ryzykownego klienta lub branży medycznej - mimo wszystko użyj preview.

SOP 4 - Zmiana stylu strony głównej

- Utwórz request redesign_homepage.
- Klasyfikacja: design_edit.
- Zmień design_profile albo page_section_variants.
- Zbuduj preview i przejdź checklistę wizualną.
- Dopiero po review publikuj.

17. Załącznik A - Minimalna schema v3

Poniższa lista nie jest pełnym SQL-em, ale jest wystarczająco konkretna, żeby zamienić ją na migracje w kolejnym kroku.

Tabela	Kluczowe pola	Uwagi
clients	id, company_name, industry, status	Główna encja klienta.
site_config	client_id, phone, email, address, template_id	Globalne dane publikacyjne i feature flags.
services	id, client_id, name, slug, is_active	Encje biznesowe usług.
locations	id, client_id, city, slug, is_main, is_active	Miasta i obszary działań.
pages	id, client_id, slug, body_markdown, is_active	Strony stałe.
client_posts	id, client_id, slug, status	Wpisy eksperckie.
asset_library	id, client_id, type, storage_path, alt_text	Jednolita biblioteka mediów.
change_requests	id, client_id, request_type, change_class, status	Centrum zmian v3.
jobs	id, client_id, job_type, status, payload_json	Kolejka procesów.
job_steps	job_id, step_name, status, output_json	Postęp etapów dużych jobów.
service_versions	service_id, version_no, status, full_description, metadata	Wersje usługi.
location_versions	location_id, version_no, status, description, metadata	Wersje lokalizacji.
page_versions	page_id, version_no, status, body_markdown, metadata	Wersje stron stałych.
post_versions	post_id, version_no, status, body_markdown, metadata	Wersje postów.
deployments	id, client_id, snapshot_id, type, status, deploy_url	Historia preview/publish/rollback.
deployment_snapshots	id, client_id, snapshot_json, template_version	Zamrożony stan builda.
design_profiles	client_id, template_family, font_pair, cta_style	Kontrolowana unikalność.

page_section_variants	client_id, route, section_key, variant_key	Warianty bloków per strona.
content_diffs	request_id, before_json, after_json, diff_summary	Podgląd zmian przed publikacją.
indexing_events	deployment_id, url, status, response	Historia indeksacji.
system_logs	client_id, log_type, message, created_at	Audyt działań i błędów.

Załącznik B - Status machine

- Client: lead -> brief_received -> intake_verified -> seo_generating -> content_review -> design_review -> build_ready -> preview_ready -> deployed -> maintenance -> archived
- Change request: new -> triaged -> draft_in_progress -> awaiting_review -> approved -> queued_for_publish -> published -> rolled_back | rejected
- Job: queued -> running -> waiting_for_input -> failed | completed | canceled
- Version: draft -> approved -> published -> superseded -> archived
- Deployment: queued -> building -> preview_ready -> published | failed | rolled_back

Załącznik C - Definition of Done dla v3 MVP

- Masz Source of Truth w Supabase i działający pierwszy template live.
- Masz dashboard z listą klientów, edycją klienta i generowaniem strony.
- Masz change_requests, jobs i co najmniej jeden worker buildowy.
- Każda zmiana tworzy log i może zostać przypisana do requestu.
- Masz wersjonowanie usług, stron albo lokalizacji przynajmniej dla najważniejszych obiektów.
- Masz snapshot i historię deploymentów.
- Masz rollback działający na podstawie starego snapshotu.
- Masz minimum jeden template family i jeden design profile działający produkcyjnie.